Backend runs on 2 IP addresses: `127.0.0.1:5001` — localhost (this Mac only)

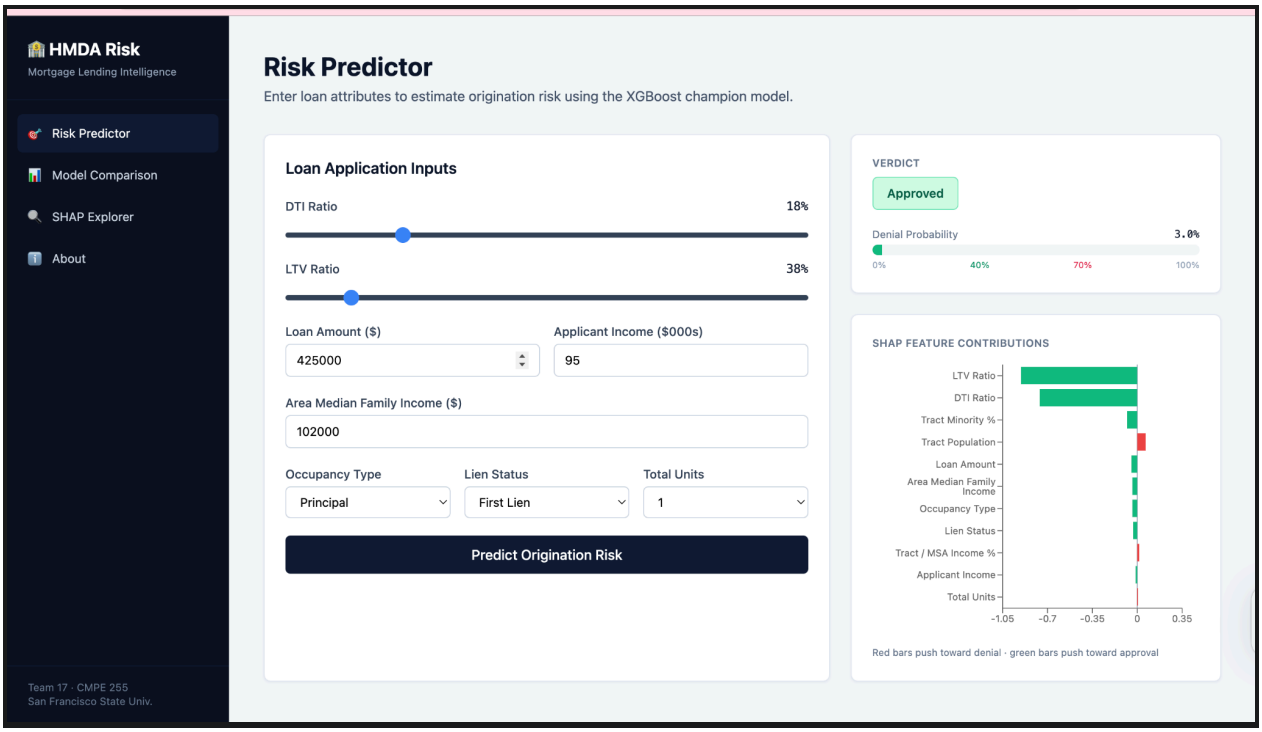
- `192.168.0.30:5001` — your LAN IP (other devices on your WiFi could also reach it)
- http://127.0.0.1:5001/health

127.0.0.1:5001/health

Pretty print


```
{
  "model_loaded": true,
  "model_path": "/Users/dineshsingh/HMDA_Mortgage_Risk/backend/xgb_model.pkl",
  "status": "ok"
}
```


Frontend runs at : (http://localhost:5173)



HMDA Risk
Mortgage Lending Intelligence

- Risk Predictor
- Model Comparison
- SHAP Explorer
- About

Team 17 · CMPE 255
San Francisco State Univ.

Model Comparison

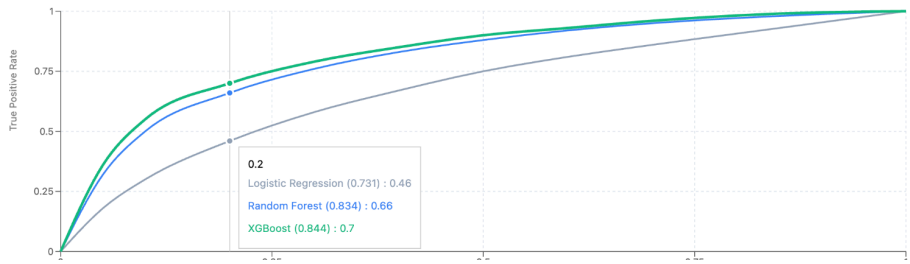
Three classifiers evaluated on the held-out HMDA 2024 California test set.

Performance Summary

MODEL	TEST AUC	DENY PRECISION	DENY RECALL	F1	NOTES
Logistic Regression	0.731	0.682	0.521	0.591	Baseline · interpretable
Random Forest (safe)	0.834	0.812	0.684	0.742	Strong ensemble
XGBoost (safe)	0.844	0.898	0.622	0.735	🏆 Champion

ROC Curves

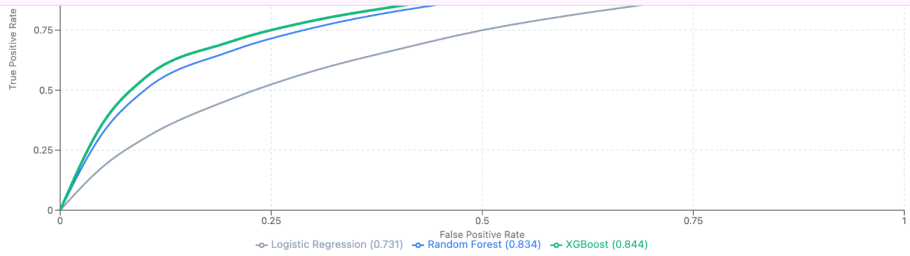
True Positive Rate vs False Positive Rate · higher area under curve (AUC) is better



localhost:5173/models

HMDA Risk
Mortgage Lending Intelligence

- Risk Predictor
- Model Comparison
- SHAP Explorer
- About



XGBoost Threshold Sensitivity

Operating point selection for the deny class. Higher thresholds raise precision but lower recall.

THRESHOLD	DENY PRECISION	DENY RECALL	F1
0.40	0.624	0.781	0.694
0.45	0.681	0.712	0.696
0.50	0.738	0.622	0.675
0.55	0.790	0.531	0.635
0.60	0.836	0.432	0.570
0.65	0.872	0.341	0.490
0.70	0.898	0.262	0.405

Highlighted row (0.70) is the deployed operating point — chosen for high deny precision (0.898) to minimize false denials.

Team 17 · CMPE 255
San Francisco State Univ.

HMDA Risk
Mortgage Lending Intelligence

- Risk Predictor
- Model Comparison
- SHAP Explorer
- About

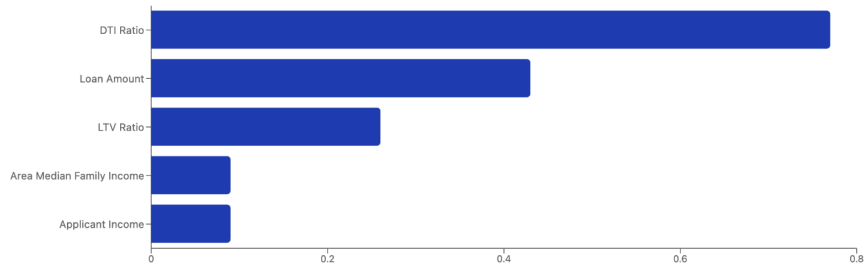
Team 17 · CMPE 255
San Francisco State Univ.

SHAP Explorer

Mean absolute SHAP values across the test set — which features drive the XGBoost model globally.

Global Feature Importance

Average absolute SHAP value · larger = more influential on the prediction



Directional Impact

DTI Ratio

`↑ DTI → ↑ denial risk`

Debt-to-income is the dominant signal in the XGBoost model. Applicants whose total monthly debts exceed roughly 43% of gross income are far more likely to be denied. The relationship is non-linear — risk accelerates above 50%.

Loan Amount

`↑ Loan amount (relative to income) → ↑ denial risk`

localhost:5173/shap

HMDA Risk
Mortgage Lending Intelligence

- Risk Predictor
- Model Comparison
- SHAP Explorer
- About

Team 17 · CMPE 255
San Francisco State Univ.

Directional Impact

DTI Ratio

`↑ DTI → ↑ denial risk`

Debt-to-income is the dominant signal in the XGBoost model. Applicants whose total monthly debts exceed roughly 43% of gross income are far more likely to be denied. The relationship is non-linear — risk accelerates above 50%.

Loan Amount

`↑ Loan amount (relative to income) → ↑ denial risk`

Large loan principals raise denial probability primarily when income does not scale with them. The model captures the interaction between loan size and applicant income implicitly via tree splits.

LTV Ratio

`↑ LTV → ↑ denial risk`

Loan-to-value above 95% materially increases denial risk because the lender has less collateral cushion. LTV near 80% (the conventional conforming threshold) is the safe zone.

Area Median Family Income

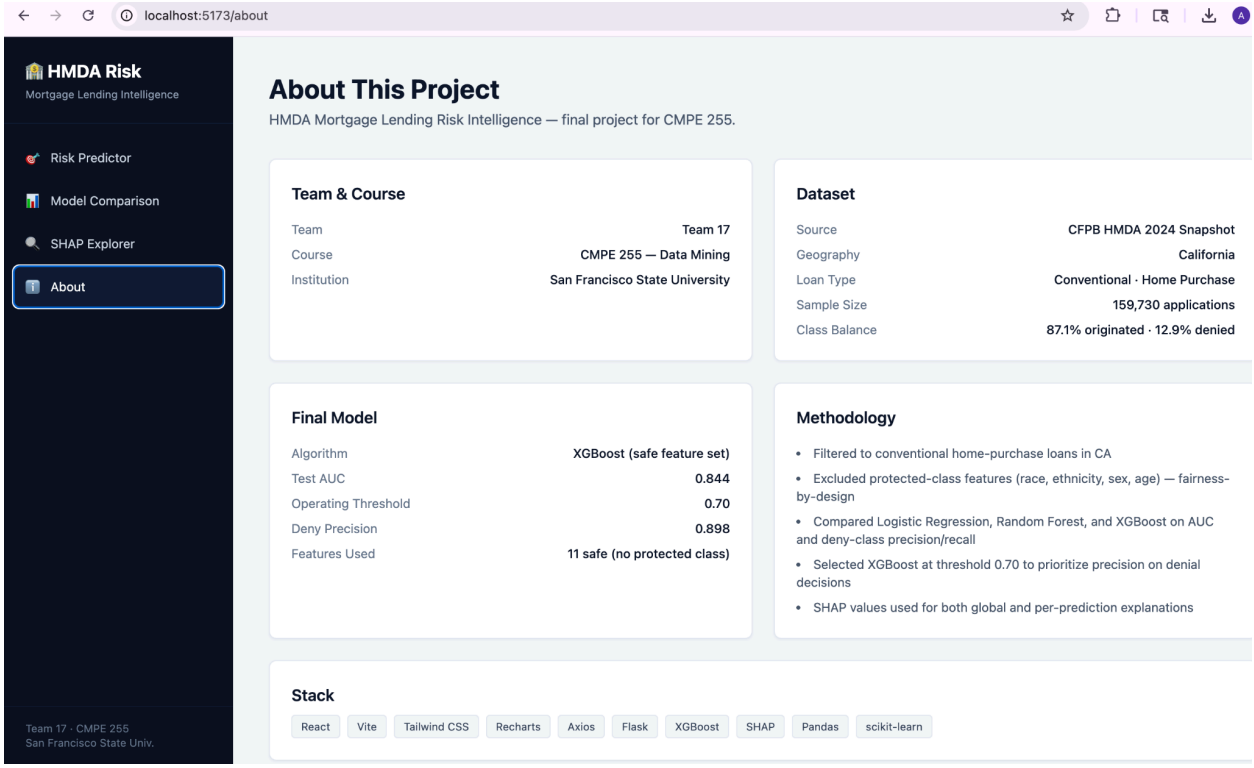
`↑ Area median income → ↓ denial risk`

Higher MSA median income correlates with stronger local lending markets and less risky underwriting outcomes. This is a tract-level economic context feature.

Applicant Income

`↑ Applicant income → ↓ denial risk`

Higher reported income reduces denial probability, especially when paired with a moderate DTI. The model interacts income with loan amount and DTI through tree-split combinations.



Here's the full picture — three pieces, one data flow.

```
┌──────────────────┐   one-time   ┌──────────────────┐   HTTP   ┌──────────────────┐
│  Google Colab    │   export     │  Flask Backend   │  POST    │  React Frontend  │
│  (training)      │ ──────────►  │  port 5001       │  ◄────   │  port 5173       │
│                  │   .pkl file  │                  │          │                  │
└──────────────────┘              └──────────────────┘          └──────────────────┘
```

1. Colab — trains the model (offline, one time)

In Colab you load HMDA 2024 California data, fit XGBoost, then dump it:

```
import joblib
joblib.dump(model, 'xgb_model.pkl')    # writes a binary file
files.download('xgb_model.pkl')        # downloads to your Mac
```

That's it for Colab. **Colab is not running during the demo.** It produced one file and now sits idle. You moved that file to backend/xgb_model.pkl — that's the only artifact crossing from Colab to your laptop.

2. Backend — loads the .pkl and serves predictions

When you run backend/app.py:

- Flask starts on `http://localhost:5001`

- On boot, app.py:39 calls `joblib.load("xgb_model.pkl")` → the trained model is now in memory
- One endpoint: `POST /predict`

What `/predict` does:

1. Receives JSON like `{"dti_num": 35, "ltv_num": 80, "loan_amount": 425000, ...}` (11 features)
2. Wraps it in a pandas DataFrame in the exact column order
3. Calls `model.predict_proba(X)` → gets the deny probability
4. Calls `booster.predict(..., pred_contribs=True)` → gets per-feature SHAP values
5. Returns JSON:

```
{
  "deny_probability": 0.23,
  "verdict": "Approved",
  "shap_values": { "dti_num": 0.12, "ltv_num": -0.04, ... }
}
```

6.

3. Frontend — collects inputs, calls backend, renders results

When you run `npm run dev` in frontend/:

- Vite serves React on `http://localhost:5173`
- User logs in, fills out the Risk Predictor form, hits Submit
- src/api/client.js:91 does:

```
axios.post('http://localhost:5001/predict', formValues)
```

-
- The response (JSON above) drives the UI:
  - `verdict` → colored badge (green/amber/red)
  - `deny_probability` → progress bar width
  - `shap_values` → red/green waterfall chart in Recharts

The simulation fallback (the safety net)

If the backend is down or unreachable, axios throws a network error. client.js:97 catches it and runs `simulatePrediction()` instead — a pure-JavaScript copy of the model logic using the SHAP weights from your training results. The amber banner appears at the top to signal "not real predictions."

That's why the UI works even when Flask isn't running — useful when WiFi flakes during a presentation.

CORS — why it's needed

Frontend (`localhost:5173`) and backend (`localhost:5001`) are different "origins" — browsers block cross-origin requests by default. app.py:33 does `CORS(app, ...)` which adds the right headers so the browser allows the call.

One-line summary

Colab made the model, Flask serves it, React calls Flask, simulation mode rescues the demo if Flask is dead.